Claim Denial Risk — Prediction & GenAI Explanations

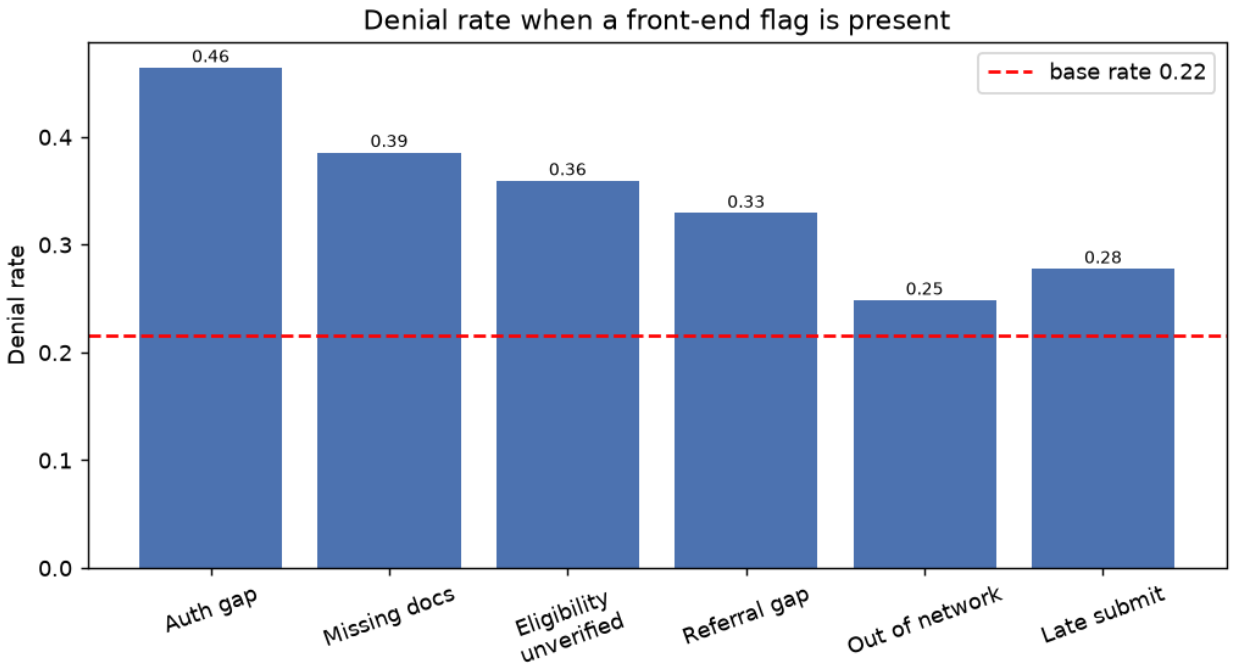
Ensemble Health Partners — AI/ML Take-Home Assessment. Binary classification to triage claims before submission, evaluated against a fixed **top-25% review capacity**.

1. Problem framing — which errors matter more

A front-end team can manually review only the **top 25%** of daily claims. So the model's job is to **rank** claims by denial risk, and success is measured by how many true denials fall inside that 25% (**denial capture @ top 25%**). A **false negative** (a claim that will be denied but is not flagged) is the expensive error: it sails through and generates downstream rework — investigate, correct, resubmit. A **false positive** only costs a few minutes of analyst review. We therefore optimise recall within the review budget rather than raw accuracy, which is misleading here (a 'deny nothing' model is ~78% accurate but catches zero denials).

2. Three findings for a non-technical manager

- **Front-end readiness drives denials.** When prior authorization is required but not on file, the denial rate is **~45%** — more than double the ~22% baseline. Missing documentation (~40%) and unverified eligibility (~35%) are the next biggest levers. These are all fixable *before* submission.
- **Who the payer is and the visit type matter.** Medicaid MCO (~31%) and Medicare Advantage (~25%) deny more than Commercial (~15%); Inpatient stays (~31%) deny far more than Outpatient/Observation (~18–20%).
- **Late claims deny more.** Claims submitted in the slowest quartile (25+ days after care) deny ~28% vs ~18% for the fastest — a timely-filing signal the team can act on by expediting at-risk claims.



Denial rate when each front-end flag is present, vs. the base rate (dashed).

3. What we built, baselines, and threshold choice

Preprocessing, feature engineering, training, evaluation, scoring, and explanation are separated into importable modules; all transforms live inside a scikit-learn `Pipeline` so train and score paths are identical. Only pre-submission fields are used; `claim_id`, `is_denied`, `denial_reason`, and `split` are excluded. Engineered features include auth/referral gaps, a readiness-deficit counter, payment ratio, late-submission flag, and log-billed.

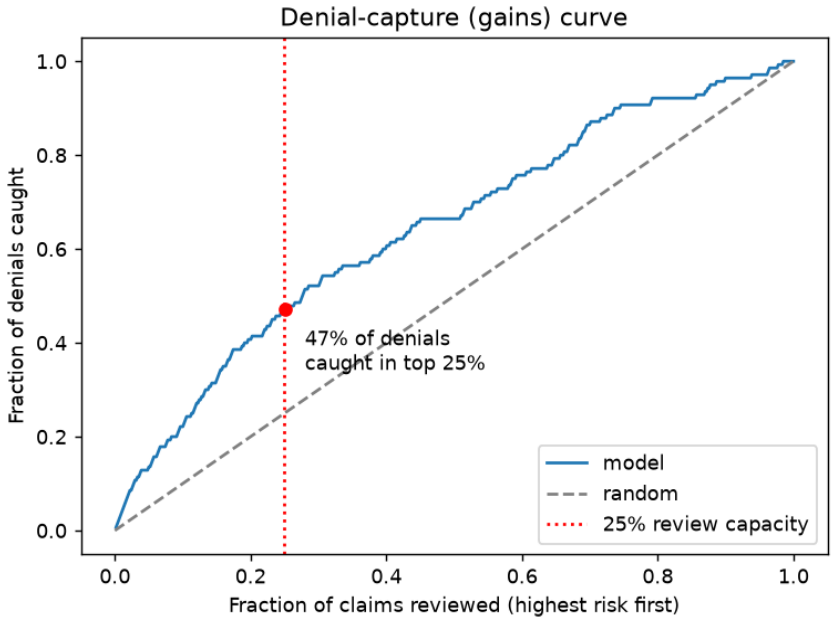
We compared a **rule baseline** (rank by number of readiness deficits) and three models. Selection metric: validation denial-capture@25%, tie-broken by PR-AUC.

Model	Capture@25% (val)	Precision@25% (val)	PR-AUC (val)	ROC-AUC (val)
Rule baseline	0.398	0.304	—	—
LogReg (selected)	0.476	0.363	0.449	0.709
RandomForest	0.476	0.363	0.398	0.674
GradientBoosting	0.456	0.348	0.391	0.679

Logistic Regression (class-weighted) was selected: it matched the best capture@25% while giving the strongest PR-AUC and a fully interpretable, well-calibrated ranking. **Threshold choice:** we set the operating threshold at the score that flags the top 25% on validation (`p = 0.59`) — deliberately tied to the team's actual review capacity rather than a generic 0.5. Risk tiers: **High** = riskiest 10%, **Medium** = next 15% (High+Medium ≈ reviewable 25%), **Low** = the rest.

4. Test-set performance and denial capture @ top 25%

On the held-out **test** split (n=539, base rate 26%): **ROC-AUC 0.696**, **PR-AUC 0.510**. Reviewing only the top 25% of claims captures **47% of all denials** (66/140) at 49% precision — about **1.9× better than random** (random review of 25% would catch ~25%). In practice the team does the same amount of work but catches nearly half of all denials before they go out.



Gains curve: denials caught vs. fraction reviewed; red line = 25% capacity.

5. GenAI explanations — prompt, example, sanity check

For the top-10 riskiest current claims, the model's grounded risk drivers feed an LLM prompt (provider-agnostic; runs on OpenAI/Anthropic when a key is set, else a grounded offline template). The prompt forces the model to use only the claim's real fields, add one concrete action, and hedge as a risk estimate. Prompt template (abridged):

> You are a claims-denial risk assistant… Use ONLY the facts below. CLAIM FACTS: id, payer, visit type, billed/expected, days-to-submit, model probability + tier. TOP RISK DRIVERS: … Write 2–3 sentences: state the risk in plain language, name the driver(s), give ONE concrete action, and make clear this is a risk estimate, not a guaranteed denial.

Example — highest-risk claim (CCLM-00082):

> This claim (CCLM-00082, payer P009) shows a high risk of denial at about 94%. The main concern is that required supporting documentation appears to be missing, and also patient eligibility was not verified before the visit and referral required but not present. Recommended action: gather the missing supporting documents and attach them before submitting. This is a model risk estimate, not a guaranteed denial.

Low-risk sanity check (CCLM-00442): the same prompt behaves sensibly — it reports low risk and a light completeness nudge instead of inventing problems:

> This claim (CCLM-00442, payer P001) shows a low risk of denial at about 9%. No single front-end problem stands out; the score reflects a combination of softer payer and visit signals. Recommended action: Do a quick completeness check on authorization, documentation, and eligibility before submitting. This is a model risk estimate, not a guaranteed denial.

Honest limitation: explanations are only as trustworthy as the risk drivers behind them. The drivers are correlational, not causal — 'eligibility unverified' flags risk but fixing the checkbox alone won't prevent a denial rooted in medical necessity. The text should guide triage, not be quoted to a payer as the reason.

6. What I'd improve with more time

- **Calibration & cost model.** Calibrate probabilities (isotonic) and pick the threshold from an explicit cost of rework vs. review time, instead of a fixed 25% cut.
- **Denial-reason modelling.** Use `denial_reason` (post-hoc only) to build per-reason models or a multiclass head, so explanations point at the *likely* denial category.
- **Richer signals & monitoring.** Add payer×procedure history, drift monitoring, and A/B measurement of prevented denials once the tool is in the analysts' workflow.
- **LLM hardening.** Add automated groundedness checks (does the text only reference given fields?) and a human-feedback loop on explanation usefulness.